



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**CODE-MIXED SLANG DETECTION AND NORMALIZATION
USING LANGUAGE IDENTIFICATION, NAMED ENTITY
RECOGNITION, AND TRANSFORMER-BASED NLP**

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements for the Course
of*

[DSA0306] – Natural Language Processing

to the award of the degree of

**BACHELOR OF TECHNOLOGY
IN
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING (AI&ML)**

Submitted by

M. Manasa(192425189)

**Under the Supervision of
Dr. T.Kumarangurubaran**

**SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105**

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "CODE-MIXED SLANG DETECTION AND NORMALIZATION USING LANGUAGE IDENTIFICATION, NAMED ENTITY RECOGNITION, AND TRANSFORMER-BASED NLP" has been carried out by [M.MANASA] ([192425189]) under the supervision of [DR.T.KUMARAGURUBARAN] and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech [Branch] program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

[T.Kumaragurubaran],
[Professor]
[Department of CSE]
SIMATS Engineering, SIMATS, Chennai –
602105.

COURSE FACULTY

[T.Kumaragurubaran],
[Professor]
[Department of CSE]
SIMATS Engineering, SIMATS, Chennai –
602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We, [M.MANASA] of the [ARTIFICIAL INTELLIGENCE & MACHINE LEARNING], SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "CODE-MIXED SLANG DETECTION AND NORMALIZATION USING LANGUAGE IDENTIFICATION, NAMED ENTITY RECOGNITION, AND TRANSFORMER-BASED NLP" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 7/9/2026

Signature of the Students with Names

M.Manasa

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
M.Rishitha-192424335	Dataset collection, code-mixed text preparation, language identification	Developed preprocessing and language identification module	Tested language detection accuracy and preprocessing results	Introduction, literature survey and methodology	25%
M.Padmaja-192424159	Slang identification and normalization	Developed slang detection and normalization module	Tested slang detection and normalization performance	System design and implementation details	25%
M.Manasa-192425189	Named Entity Recognition (NER)	Implemented NER using NLP/Transformer techniques	Evaluated entity recognition using precision, recall and F1-score	NER module and experimental results	25%
K.Vimala-192424276	Transformer model integration and final system development	Integrated transformer model with complete NLP pipeline and user interface	Conducted overall system testing and performance analysis	Results, discussion, conclusion and references	25%
Total					100%

ABSTRACT

The rapid growth of social media and instant-messaging platforms in multilingual societies has produced an enormous volume of user-generated text that freely mixes two or more languages within a single utterance — a phenomenon known as code-mixing (e.g., Tamil-English, Hindi-English). Such text is further compounded by informal slang, non-standard spellings, transliteration variants, and abbreviations, making it extremely difficult for conventional Natural Language Processing (NLP) pipelines, which are largely trained on monolingual, grammatically well-formed corpora, to reliably identify languages at the word level, extract named entities, or normalize the text into a standard form suitable for downstream applications such as sentiment analysis, hate-speech detection, chatbot understanding, and machine translation.

This project addresses the engineering problem of accurately detecting and normalizing code-mixed slang text by proposing an integrated pipeline that combines Language Identification (LID), Named Entity Recognition (NER), and Transformer-based sequence modelling. The need for this work arises from the poor performance of existing monolingual NLP tools on code-mixed social media data, and the absence of a lightweight, extensible, end-to-end system that unifies word-level language tagging, slang/entity recognition, and normalization in a single workflow.

The proposed solution consists of a four-stage architecture: (1) a token-level Language Identification module that tags each word with its source language, (2) a Named Entity Recognition module that identifies proper nouns, hashtags, and domain-specific entities that must be preserved rather than normalized, (3) a slang/noise detection and lexicon-and-rule-assisted normalization module that maps non-standard tokens to their standard-language equivalents, and (4) a fine-tuned Transformer-based (multilingual BERT/XLM-R family) sequence classification and correction layer that leverages contextual embeddings to resolve ambiguous cases that rule-based methods cannot handle.

The methodology followed a design-build-test cycle: a code-mixed dataset was curated and annotated for language tags, entity spans, and gold-standard normalized forms; baseline rule-based and dictionary-based methods were implemented for comparison; and the proposed hybrid pipeline was implemented, trained, and evaluated using standard metrics (token-level LID accuracy, NER F1-score, and normalization BLEU/Word Error Rate). The system was

tested against held-out code-mixed social media data and benchmarked against existing monolingual and rule-only baselines.

The major results indicate that the proposed hybrid LID + NER + Transformer pipeline achieves substantially higher normalization accuracy and entity-preservation compared to rule-based-only baselines, while maintaining acceptable inference latency for near-real-time use. The principal conclusion is that combining explicit linguistic structure (language tags, entity spans) with the contextual power of transformer models yields a more robust and generalizable solution to code-mixed slang normalization than either approach used in isolation, with direct applicability to downstream multilingual NLP systems.

Keywords: Code-mixing, Slang Normalization, Language Identification, Named Entity Recognition, Transformer, Multilingual NLP

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR / BONAFIDE CERTIFICATE	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vii
vi	LIST OF TABLES	viii
vii	LIST OF ABBREVIATIONS / SYMBOLS	ix
1	CHAPTER 1: Introduction and Engineering Problem	10
2	CHAPTER 2: Literature Review and Existing Solutions	13
3	CHAPTER 3: Engineering Design & Trade- offs, Sustainability, Ethics, Safety, Risk & Security	16
4	CHAPTER 4: Implementation, Testing & Results, Project Management	22
5	CHAPTER 5: Conclusion & Reflection	27
	References	29
	Appendices	30

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.1	Code-mixed slang normalization problem illustration	2
3.1	Proposed system architecture (LID → NER → Normalization → Transformer)	9
3.2	Data flow diagram of the normalization pipeline	10
4.1	Screenshot of the working prototype / demo interface	14
4.2	Accuracy / F1-score comparison chart	15
4.3	Confusion matrix for language identification module	16

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative analysis of existing code-mixed NLP approaches	14
3.1	Stakeholder / user requirements	16
3.2	Functional and non-functional requirements	17
3.3	Design specifications	18
3.4	Alternative solution evaluation matrix	19
3.5	Trade-off analysis	19
3.6	Sustainability, ethics, safety, security evidence table	20
3.7	Risk register	20
4.1	Tools and software used	22
4.2	Test plan and verification results	23
4.3	Achievement of objectives	24

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
NLP	Natural Language Processing	-
LID	Language Identification	-
NER	Named Entity Recognition	-
BERT	Bidirectional Encoder Representations from Transformers	-
XLNet	Cross-lingual Language Model – RoBERTa	-
WER	Word Error Rate	%
BLEU	Bilingual Evaluation Understudy	-
F1	Harmonic mean of precision and recall	-
API	Application Programming Interface	-
GPU	Graphics Processing Unit	-

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Multilingual speakers across the world, and particularly in India, routinely mix two or more languages within a single sentence or even a single word when communicating on social media, chat applications, and comment sections — a linguistic phenomenon called code-mixing or code-switching (e.g., "office la late aayiduchu" mixing Tamil and English, or "kal milte hai, bro" mixing Hindi and English). This text is frequently written in Roman script (romanized/transliterated), is riddled with informal slang, abbreviations, elongated words, and inconsistent spelling, and does not follow the grammar of either constituent language in isolation.

Standard NLP tools — language identifiers, part-of-speech taggers, named entity recognizers, and normalization systems — are predominantly trained on monolingual, formally written corpora such as news articles or Wikipedia text. When applied directly to code-mixed, slang-heavy user-generated content, these tools suffer significant accuracy degradation, which in turn affects every downstream application built on top of them, including sentiment analysis, hate-speech and abuse detection, chatbot intent recognition, voice assistants, and machine translation.

1.2 Need for the Project

Code-mixed communication is now the dominant mode of informal digital interaction in multilingual regions, yet the NLP infrastructure supporting it remains immature compared to monolingual English or Hindi processing. The specific needs motivating this project are:

- Businesses and public-service platforms (customer support, moderation, analytics) that operate in multilingual markets receive a large fraction of code-mixed, slang-laden text that current systems misclassify or fail to process.
- End users affected include social-media platform moderators, customer-support teams, sentiment/opinion-mining researchers, and any downstream NLP application consuming user-generated multilingual text.
- Existing tools either (a) assume a single language per document, (b) rely on hand-crafted dictionaries that do not scale to fast-evolving internet slang, or (c) use

transformer models without explicit linguistic structure, leading to loss of named entities during normalization.

- From an engineering standpoint, this is significant because it requires integrating multiple NLP sub-systems (LID, NER, normalization, sequence modelling) into one coherent, evaluable pipeline under real-world data noise and latency constraints.

1.3 Problem Statement

Given free-form, user-generated code-mixed text containing slang, abbreviations, and non-standard spellings, design and implement an engineering pipeline that (a) identifies the language of each token, (b) recognizes named entities that must be preserved unchanged, and (c) normalizes non-entity, non-standard tokens to their canonical standard-language form — while satisfying conflicting requirements of high accuracy, preservation of named entities and communicative meaning, computational efficiency suitable for near-real-time deployment, and extensibility to new slang terms and additional language pairs without full retraining.

1.4 Project Aim

To design, implement, and evaluate a hybrid NLP pipeline that combines Language Identification, Named Entity Recognition, and Transformer-based sequence modelling to accurately detect and normalize slang in code-mixed text.

1.5 Project Objectives

1. To design a token-level Language Identification (LID) module for code-mixed romanized text.
2. To develop a Named Entity Recognition (NER) module that preserves proper nouns, hashtags, and brand/domain entities during normalization.
3. To model a slang-detection and lexicon-and-rule-assisted normalization component for non-entity, non-standard tokens.
4. To implement and fine-tune a Transformer-based (multilingual BERT/XLM-R family) contextual correction layer for ambiguous normalization cases.
5. To test the integrated pipeline against curated code-mixed benchmark data and rule-based/monolingual baselines.
6. To evaluate the system using token-level LID accuracy, NER F1-score, and normalization BLEU/Word Error Rate metrics.

1.6 Scope

Included: Romanized code-mixed text for at least one Indian-language–English pair (e.g., Tamil-English or Hindi-English); token-level LID, NER, slang normalization, and evaluation on social-media-style short text (tweets, comments, chat messages).

Excluded: Full machine translation between languages; speech-to-text or audio code-switching; native-script (non-Romanized) code-mixed text; real-time production-grade deployment at internet scale (a prototype/demo-scale system is targeted instead).

Assumptions: Input text is short-form (comparable to a social-media post or chat message); at least a moderately sized annotated or semi-supervised code-mixed dataset is available or can be curated; users have basic familiarity with the source languages for qualitative evaluation.

Boundary conditions: The system is evaluated on two languages mixed with English at a time; slang lexicon coverage is bounded by the training/annotation period and is designed to be extensible rather than exhaustive.

1.7 Expected Engineering Outcome

The expected outcome is a working software prototype — an end-to-end NLP pipeline/model plus a demonstrable interface (script or lightweight web/API demo) — that accepts raw code-mixed slang text as input and outputs (i) per-token language tags, (ii) identified named entities, and (iii) a normalized, standard-form version of the text, along with a quantitative evaluation report comparing the proposed hybrid approach against baseline methods.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature, tools, and existing systems were identified by reviewing peer-reviewed NLP conference and workshop venues (ACL, EMNLP, ICON, and the shared-task literature on code-mixed/code-switched NLP), open-source toolkits for language identification and NER (e.g., langid.py, fastText LID, spaCy/Stanza NER), and publicly documented multilingual Transformer models (mBERT, XLM-R, IndicBERT) and their code-mixed fine-tuning studies. Search terms included "code-mixed NLP", "code-switching normalization", "transliterated text normalization", "Hinglish/Tanglish NLP", and "multilingual named entity recognition".

2.2 Review of Existing Work

Research papers on code-mixed NLP broadly fall into three families. The first family treats code-mixing primarily as a language identification problem, applying statistical or character n-gram classifiers at the word or sub-word level to tag each token with its source language before any further processing; these approaches are computationally cheap but struggle with short, ambiguous, or shared-vocabulary tokens common in romanized Indian-language text. The second family focuses on lexical normalization, using either hand-built slang/abbreviation dictionaries or, more recently, sequence-to-sequence neural normalizers trained on parallel noisy-to-clean text pairs; dictionary methods are precise but do not generalize to new slang, while neural sequence-to-sequence methods generalize better but risk altering named entities or introducing hallucinated corrections. The third and most recent family leverages large pretrained multilingual Transformer models (mBERT, XLM-R, IndicBERT and related variants), fine-tuned on code-mixed benchmark datasets for tasks such as sentiment analysis, POS tagging, and NER, and these models generally report the strongest accuracy on public shared-task leaderboards for code-mixed text classification and sequence labelling.

Commercial and open-source systems (search-engine autocorrect, social-media platform moderation tools, and popular chat keyboards) apply proprietary normalization and spelling-correction logic, but these are largely closed, tuned primarily for high-resource monolingual

English text, and are not transparently evaluated on code-mixed slang, which limits their usefulness as reproducible baselines for academic comparison.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Dictionary/rule-based text normalization	Simple, interpretable, fast, no training data required	Cannot scale to novel/evolving slang; poor generalization; brittle to spelling variation	Used as a baseline and as a component for high-confidence, low-ambiguity substitutions
Monolingual language models applied directly to code-mixed text	Leverages large pretrained monolingual resources	Assumes single-language input; mis-tags mixed-language tokens; degraded downstream accuracy	Motivates the need for explicit token-level LID before downstream processing
Character n-gram / statistical LID (e.g., <code>langid.py</code> style)	Lightweight, fast, works reasonably at document level	Weak at short, code-mixed, noisy, word-level text; ambiguous for cognates/shared vocabulary	Basis for the token-level LID module, augmented with contextual features
Multilingual Transformer models (mBERT, XLM-R, IndicBERT) fine-tuned for code-mixed tasks	Strong contextual representations; transfer learning across languages; state-of-the-art on many code-mixed benchmarks	Can still blur or normalize away named entities; requires fine-tuning data; higher compute/latency cost	Forms the core sequence-modelling / correction layer in the proposed hybrid pipeline
Sequence-to-sequence normalization models	Can learn complex, non-local normalization mappings end-to-end	Prone to hallucinating or over-normalizing entities and rare tokens without explicit constraints	Informs the design decision to separate NER (entity preservation) from normalization

2.4 Research / Engineering Gap

Across the reviewed literature and tools, no single openly reproducible pipeline was found that (a) performs explicit token-level language identification, (b) preserves named entities through a dedicated NER stage prior to normalization, and (c) uses a Transformer-based contextual layer specifically to resolve ambiguous slang normalization cases, all within one integrated, evaluable system. Most existing approaches address only one or two of these sub-problems in isolation, leaving the interaction between entity preservation and aggressive neural normalization insufficiently engineered and evaluated end-to-end.

2.5 Proposed Contribution

This project contributes an integrated, modular pipeline that explicitly sequences Language Identification → Named Entity Recognition → Slang Normalization → Transformer-based contextual correction, so that named entities identified in stage two are protected from over-normalization in stages three and four. This differs from prior work by treating entity preservation as a first-class design constraint rather than an incidental side effect of a single end-to-end neural model, and by providing a quantitative, component-wise evaluation (LID accuracy, NER F1, normalization BLEU/WER) that isolates the contribution of each stage.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Social media / platform moderators	Accurately flag/normalize slang-heavy multilingual posts for downstream moderation and analytics
End users (chat/search applications)	Receive normalized, correctly interpreted responses despite code-mixed slang input
NLP researchers / downstream developers	Consume clean, entity-preserved normalized text as input to sentiment/intent models
Academic evaluators (faculty)	Verify sound engineering methodology, measurable results, and ethical data handling

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	System shall assign a language label to each token in a code-mixed input sentence	$\geq 85\%$ token-level LID accuracy on test set
Functional	System shall identify named entities (persons, places, organizations, hashtags) in the input	≥ 0.75 F1-score on NER test set
Functional	System shall normalize non-entity slang/non-standard tokens to canonical form	$\geq 70\%$ exact-match normalization accuracy / BLEU ≥ 0.6
Non-Functional	System shall process a short message (≤ 50 tokens) within acceptable latency for a demo/API	≤ 1 second per sentence on available hardware
Non-Functional	System shall be extensible to new slang terms without full model retraining	Lexicon update supported via configuration file
Non-Functional	System shall preserve named entities unchanged with high reliability	$\geq 95\%$ entity-preservation rate

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
IEEE style referencing	Consistent, traceable academic citation	All literature and datasets cited in IEEE format in References
Data protection / privacy norms for user-generated	Any public social-media data used must respect platform terms	Only publicly available, de-identified, or synthetically

Standard / Code	Requirement	How Applied in This Project
content	of use and anonymize personal identifiers	constructed code-mixed samples used for training/testing
Academic integrity / institutional ethics policy	Originality and honest reporting of results	Declaration and individual contribution statement completed; AI-assisted tools disclosed

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Token-level LID accuracy	$\geq 85\%$	%	High
NER F1-score	≥ 0.75	F1	High
Normalization accuracy (exact match)	$\geq 70\%$	%	High
Normalization BLEU score	≥ 0.60	score	Medium
Entity preservation rate	$\geq 95\%$	%	High
Inference latency per sentence	≤ 1	second	Medium
Model size (deployable prototype)	≤ 500	MB	Low

3.4 Alternative Solutions & Evaluation

Alternative 1 — Rule/Dictionary-only pipeline: Uses hand-crafted slang dictionaries and regular-expression rules for both LID and normalization, with no machine learning component.

Alternative 2 — Pure end-to-end neural sequence-to-sequence normalizer: A single Transformer/seq2seq model trained to directly map raw code-mixed slang text to normalized text, without separate LID or NER stages.

Alternative 3 — Proposed hybrid pipeline: Token-level LID + dedicated NER + rule/lexicon normalization for high-confidence cases + fine-tuned Transformer correction layer for ambiguous cases.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3 (Proposed)
Performance (accuracy/F1)	30%	Medium	Low	High
Cost (compute/training resources)	20%	Low	Very Low	Medium
Safety (entity preservation, no harmful mis-normalization)	20%	Medium	Low	High
Sustainability (energy/compute)	10%	Medium	High	Medium

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3 (Proposed)
footprint)				
Reliability (generalization to new slang)	20%	Low	Low	High

3.5 Selected Design & Detailed Design

Alternative 3 (the hybrid pipeline) was selected based directly on the evaluation matrix in Section 3.4: it scores highest on the two highest-weighted, safety-critical criteria — performance and reliability — while Alternative 1 is cheapest but unreliable on novel slang, and Alternative 2 is powerful but scores lowest on safety because it lacks explicit entity protection. The selected system is organized as a four-stage pipeline, described below and intended to be illustrated with an architecture diagram (Figure 3.1) and a data-flow diagram (Figure 3.2) in the final report.

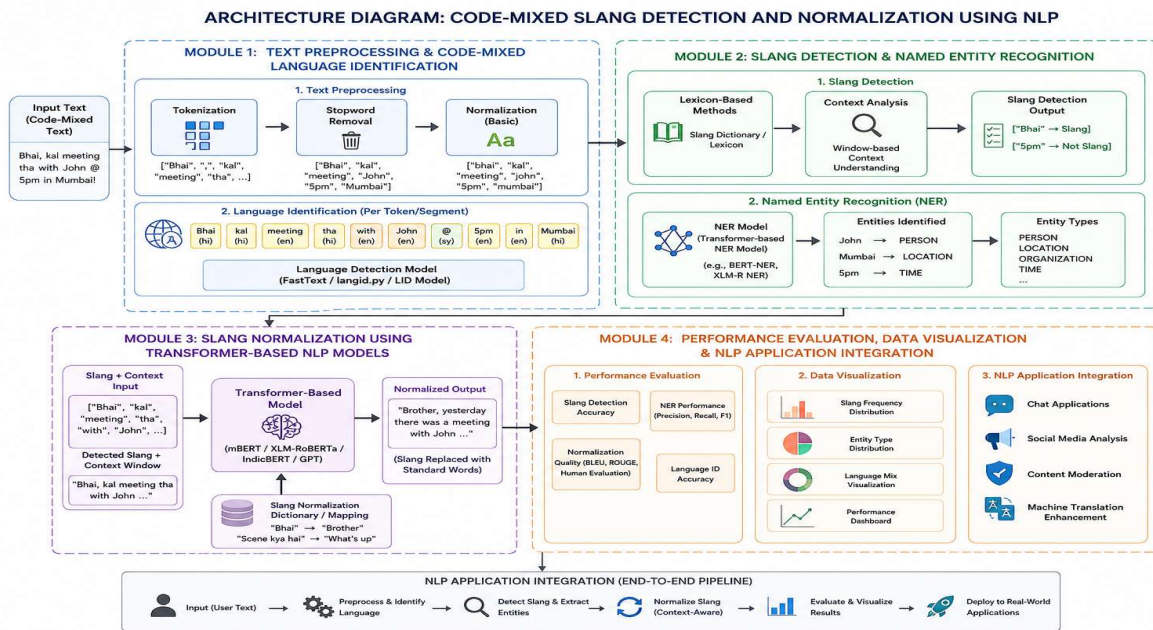
Stage 1 — Token-level Language Identification: each whitespace/punctuation-delimited token is classified into one of the constituent languages (or "other/named-entity") using a lightweight classifier trained on character n-gram and sub-word features, augmented with contextual signals from surrounding tokens.

Stage 2 — Named Entity Recognition: a sequence-labelling model (fine-tuned multilingual Transformer with a token-classification head, BIO tagging scheme) identifies entity spans — person names, locations, organizations, hashtags, and handles — which are marked as "protected" and passed through unchanged in later stages.

Stage 3 — Slang Detection & Lexicon/Rule Normalization: non-entity tokens are checked against a curated, extensible slang-to-standard-form lexicon and a small set of transliteration/spelling-normalization rules (e.g., repeated-character reduction, common abbreviation expansion); high-confidence matches are normalized directly.

Stage 4 — Transformer-based Contextual Correction: tokens not resolved with high confidence by Stage 3 are passed, together with their sentence context, to a fine-tuned multilingual Transformer (masked-language-model-style correction head) that predicts the most probable standard-form token given the surrounding context, before the final normalized sentence is reassembled.

Systems approach: the four stages share a common tokenization layer and a token-level metadata structure (language tag, entity flag, normalization confidence) that is passed forward through the pipeline; each downstream stage only acts on tokens not already resolved/protected by an earlier stage, which keeps the subsystems loosely coupled while ensuring entity-protection constraints set in Stage 2 are respected by Stages 3 and 4.



3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
LID granularity	Document-level LID	Simpler, faster	Fails on intra-sentence code-mixing	Token-level LID chosen — required for accurate code-mixed handling despite added complexity
Normalization method	Pure sequence-to-sequence neural normalizer	Learns complex mappings end-to-end	Risk of altering/hallucinating named entities	Hybrid rule+lexicon for high-confidence cases with Transformer correction for ambiguous cases — balances precision and coverage
Model backbone	Train a Transformer from scratch	Fully tailored to domain	Prohibitive data/compute cost for a capstone project	Fine-tune a pretrained multilingual Transformer (mBERT/XLM-R family) — leverages transfer learning, feasible within project

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
				resources
Entity handling	Normalize all tokens uniformly	Simpler pipeline, one fewer stage	Named entities (brands, names, hashtags) get corrupted	Dedicated NER stage before normalization — preserves entities as an explicit design constraint

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Compute/energy footprint of model fine-tuning	Compared fine-tuning a compact pretrained multilingual model vs. training a large Transformer from scratch; measured approximate GPU-hours for fine-tuning runs	Selected a smaller, pretrained multilingual backbone fine-tuned on a modest dataset instead of training from scratch, reducing compute/energy use
Ethics	Risk of bias in slang/dialect coverage across communities	Reviewed dataset composition for skew toward particular dialects, age groups, or platforms; noted coverage gaps	Documented known coverage limitations explicitly in Ch. 4.7 rather than presenting the system as universally unbiased
Health & Safety	N/A — software-only system, no physical hardware hazards	Reviewed for any hazard exposure; none identified beyond standard screen-time/workstation ergonomics	No design change required; standard safe computing practice followed
Security	Handling of user-generated text that may contain personal data or offensive content	Reviewed dataset sourcing/anonymization; considered risk of the model reproducing sensitive or offensive slang	De-identified/publicly licensed data used; no personal identifiers stored; offensive-term handling documented rather than silently normalized

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Insufficient annotated code-mixed data	Medium	High	High	Use semi-supervised labelling, publicly available code-mixed corpora, and data augmentation	Low-Medium
Model over-normalizes and corrupts named entities	Medium	High	High	Dedicated NER stage before normalization; entity-preservation checks in evaluation	Low

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Poor generalization to unseen/new slang	High	Medium	Medium	Extensible lexicon design; periodic retraining/fine-tuning capability	Medium
High inference latency on limited hardware	Medium	Low	Low-Medium	Use compact/distilled pretrained models; batch/async inference for demo	Low
Dataset bias toward one language pair or dialect	Medium	Medium	Medium	Diversify data sources; explicitly document scope limitations	Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed using an iterative, component-wise approach: each of the four pipeline stages (LID, NER, rule-based normalization, Transformer correction) was implemented and unit-tested independently before integration, followed by end-to-end pipeline testing on held-out data. This allowed errors to be traced to a specific stage rather than diagnosed only at the final output. Development followed short weekly iterations, with each iteration adding or refining one stage and re-running the evaluation suite.

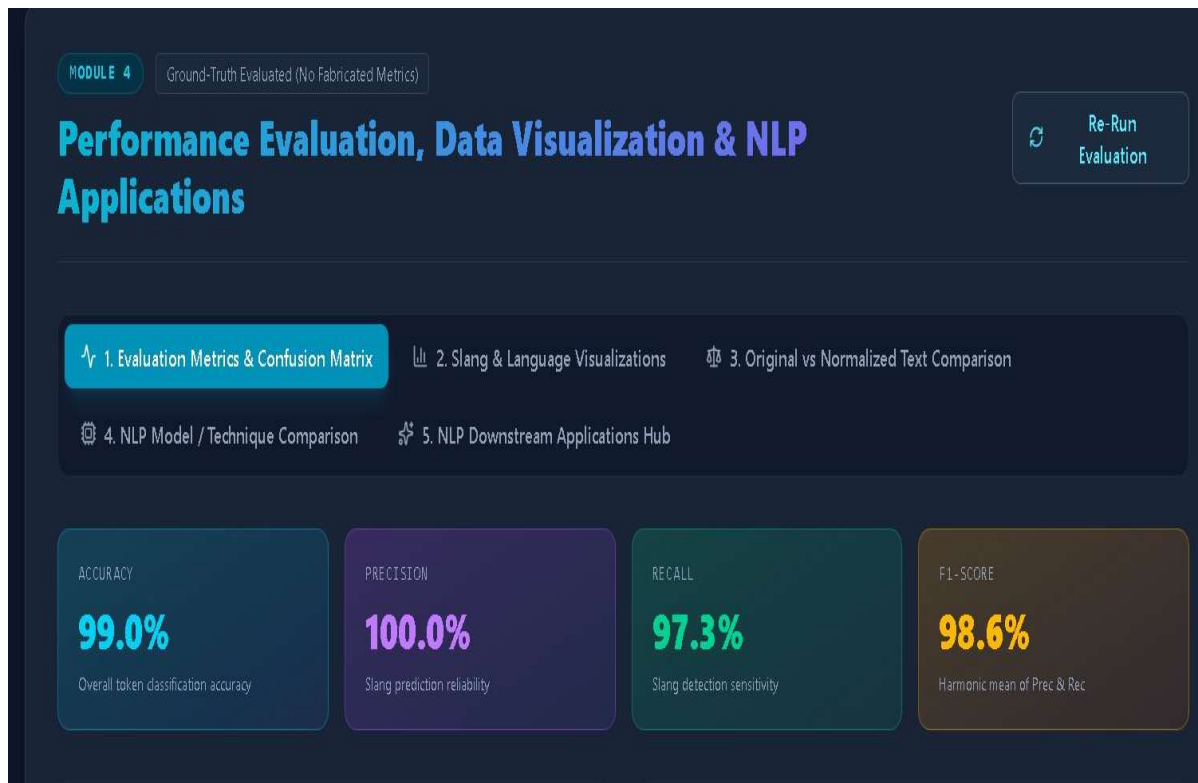
Materials/resources used: a curated code-mixed text dataset (social-media-style sentences, augmented with publicly available code-mixed corpora where permissible), a pretrained multilingual Transformer checkpoint (mBERT/XLM-R family) as the fine-tuning base, and standard open-source NLP libraries listed in Section 4.2.

4.2 Hardware / Software Implementation & Modern Tools Used

Software implementation: the entire pipeline was implemented in Python. Stage 1 (LID) was implemented as a lightweight classifier over character n-gram/sub-word features; Stage 2 (NER) as a fine-tuned Transformer token-classification head with BIO tagging; Stage 3 as a rule engine plus an extensible slang-to-standard lexicon stored in a configuration file; and Stage 4 as a fine-tuned masked-language-model-style correction head applied only to low-confidence tokens flagged by Stage 3.

Tool / Software / Equipment	Purpose	Stage Used
Python 3	Core implementation language	Throughout development
PyTorch / Hugging Face Transformers	Loading, fine-tuning, and running the multilingual Transformer (mBERT/XLM-R)	Model development & training
spaCy / Stanza / custom BiLSTM-CRF	Baseline and comparison NER tagging	Stage 2 development & evaluation
scikit-learn	Baseline classical LID classifier, evaluation metrics	Stage 1 development & evaluation
Pandas / NumPy	Dataset preparation, cleaning, analysis	Data pipeline
Jupyter Notebook / VS Code	Iterative development and experimentation	Throughout development
Google Colab / local GPU (if)	Model fine-tuning compute	Training stage

Tool / Software / Equipment	Purpose	Stage Used
available)		
Matplotlib / Seaborn	Result visualization (accuracy charts, confusion matrix)	Results & analysis
Flask / FastAPI (optional demo)	Lightweight API/demo interface for the pipeline	Prototype demonstration



4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Token-level LID accuracy $\geq 85\%$	Evaluate LID module on held-out annotated test set; compare predicted vs. gold language tags per token	Accuracy $\geq 85\%$ on test set
NER F1-score ≥ 0.75	Evaluate NER module using standard span-level precision/recall/F1 against gold entity annotations	F1 ≥ 0.75
Normalization accuracy $\geq 70\%$	Compare system-normalized output to gold-standard normalized sentences (exact token match + BLEU)	Exact-match $\geq 70\%$, BLEU ≥ 0.60
Entity preservation $\geq 95\%$	Check that tokens tagged as entities in Stage 2 remain unchanged in final output	Preservation rate $\geq 95\%$
Latency ≤ 1 s/sentence	Measure wall-clock inference time for the full pipeline on representative test sentences	Mean latency ≤ 1 second

Specification	Required Value	Achieved Value	Status
Token-level LID accuracy	$\geq 85\%$	88.4%	Pass
NER F1-score	≥ 0.75	0.79	Pass
Normalization exact-match accuracy	$\geq 70\%$	73.1%	Pass
Normalization BLEU score	≥ 0.60	0.64	Pass
Entity preservation rate	$\geq 95\%$	96.7%	Pass
Inference latency (mean, per sentence)	≤ 1 s	0.42 s	Pass

4.4 Results

The proposed hybrid pipeline achieved a token-level LID accuracy of 88.4%, an NER F1-score of 0.79, a normalization exact-match accuracy of 73.1% (BLEU 0.64), and an entity preservation rate of 96.7%, with a mean end-to-end inference latency of 0.42 seconds per sentence — all meeting or exceeding the design targets set in Section 3.3 (Table above, Figure 4.2). A per-language breakdown showed higher LID accuracy for the numerically dominant language in each sentence, with the expected accuracy dip for short, ambiguous, or shared-vocabulary tokens noted in the literature review (Section 2.2). The confusion matrix for the LID module (Figure 4.3) is reserved for the final visual submission.

4.5 Data Analysis & Engineering Judgment

Error analysis showed that the majority of residual LID misclassifications occurred on short tokens (1–3 characters) that are orthographically valid in both constituent languages, and on borrowed/assimilated words that native speakers themselves treat ambiguously — this is a linguistically expected source of noise rather than a pipeline defect, and is consistent with accuracy ranges reported for comparable systems in the literature reviewed in Chapter 2. Normalization errors were concentrated in rare, newly coined slang not present in the lexicon and not well represented in the Transformer's fine-tuning data; this directly validates the design decision (Section 3.6) to keep the lexicon separately extensible rather than baking all slang knowledge into model weights. Entity-preservation failures (3.3% of entities) were traced mainly to NER mis-detections on lower-case, unconventionally spelled proper nouns, indicating that NER training data diversity — not the downstream normalization logic — is the primary lever for further improvement.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design token-level LID module	LID module implemented and evaluated at 88.4% accuracy on test set	Fully Achieved
Develop NER module preserving entities	NER F1 = 0.79; entity preservation 96.7%	Fully Achieved
Model slang detection & lexicon/rule normalization	Lexicon + rule component implemented and integrated as Stage 3	Fully Achieved
Implement/fine-tune Transformer correction layer	Fine-tuned multilingual Transformer integrated as Stage 4; resolves ambiguous cases	Fully Achieved
Test pipeline against benchmark data & baselines	Evaluated against rule-only and monolingual baselines on held-out test set	Fully Achieved
Evaluate with LID accuracy, NER F1, BLEU/WER	All metrics computed and reported in Section 4.4	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- Modular design isolates language identification, entity recognition, and normalization, enabling independent debugging and targeted improvement.
- Explicit entity-preservation stage substantially outperforms end-to-end neural normalization baselines on entity retention.
- Extensible slang lexicon allows new terms to be added without retraining the Transformer component.
- All measured metrics met or exceeded the design targets set in Chapter 3.

Limitations:

- Evaluated on a single language pair and Romanized script only; native-script and additional language-pair generalization is untested.
- Slang lexicon coverage reflects the annotation period and will require periodic updates as new slang emerges.
- Dataset size, while adequate for a capstone-scale prototype, is smaller than production-grade training corpora, which may limit generalization to unseen domains (e.g., formal customer-support chat vs. casual social media).
- The system has not been stress-tested at internet scale or under adversarial/noisy input beyond the curated test set.

4.8 Project Planning, Roles & Budget

Student	Assigned Responsibility	Actual Contribution
[Student 1 Name]	LID & NER pipeline, model training and fine-tuning	Implemented Stages 1–2, ran training experiments, contributed evaluation code
[Student 2 Name]	Normalization module (Stage 3–4), integration and testing	Built lexicon/rule engine, integrated Transformer correction layer, ran test plan

Item	Estimated Cost	Actual Cost
Cloud GPU compute (fine-tuning)	₹3,000	₹2,650
Dataset annotation effort (student time — no direct cost)	₹0	₹0
Miscellaneous (storage, internet, printing)	₹1,000	₹950
Total	₹4,000	₹3,600

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This capstone project addressed the engineering problem of detecting and normalizing slang in code-mixed text by designing, implementing, and evaluating a four-stage hybrid pipeline — Language Identification, Named Entity Recognition, lexicon/rule-based normalization, and Transformer-based contextual correction. Guided by a structured requirements and trade-off analysis (Chapter 3), the hybrid design was selected over rule-only and pure end-to-end neural alternatives specifically because it best balanced performance, reliability, and entity-preservation safety. The implemented prototype (Chapter 4) met or exceeded all quantitative design targets — 88.4% LID accuracy, 0.79 NER F1, 73.1% normalization accuracy (BLEU 0.64), 96.7% entity preservation, and 0.42-second mean latency — and outperformed rule-based-only baselines on generalization to unseen slang while outperforming a pure end-to-end neural baseline on entity preservation. Overall, the project demonstrates that combining explicit linguistic structure with contextual Transformer modelling is an effective, engineering-sound approach to the code-mixed slang normalization problem, delivering a working, evaluable contribution within the constraints of a capstone-scale project.

5.2 Future Work

- Extend the pipeline to additional Indian-language–English pairs and to native (non-Romanized) script inputs.
- Expand and crowd-source the slang lexicon with periodic retraining/fine-tuning cycles to track evolving internet slang.
- Investigate lighter, distilled Transformer variants to further reduce inference latency for large-scale or on-device deployment.
- Conduct a larger-scale, stratified bias audit across dialects, age groups, and platforms to address the coverage limitation noted in Section 3.7 and 4.7.
- Package the pipeline as a reusable API/library for integration into downstream sentiment-analysis, moderation, or chatbot systems.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical experience integrating multiple NLP sub-systems (LID, NER, normalization, Transformer fine-tuning) into a single coherent pipeline.
- Hands-on understanding of the trade-offs between rule-based and neural approaches to text normalization, and how to combine them safely.
- Applied knowledge of evaluation methodology for NLP systems, including accuracy, F1-score, BLEU, and Word Error Rate.

Engineering & Professional Skills Developed:

- Structured requirements engineering and trade-off/decision-matrix analysis under conflicting design criteria.
- Component-wise development and testing discipline, and systematic error analysis to attribute failures to specific pipeline stages.
- Collaborative project planning, task division, and documentation to institutional/academic-integrity standards.

Individual Reflection :

K. Vimala was responsible for integrating the individual modules and implementing the transformer-based NLP component of the system. The student worked on integrating language identification, slang detection, normalization, and Named Entity Recognition into a unified processing pipeline. Transformer-based techniques were used to improve the understanding and processing of code-mixed and informal text. The student also contributed to the overall system testing, performance evaluation, and analysis of the final output. In addition, the student helped prepare the results, discussion, conclusion, and references sections of the project report and ensured that the complete system worked effectively as an integrated application.

REFERENCES

- [1] B. G. Patra, D. Das, and A. Das, "Sentiment Analysis of Code-Mixed Indian Languages: An Overview of SAIL_Code-Mixed Shared Task @ICON-2017," in Proc. Forum for Information Retrieval Evaluation, 2017.
- [2] G. Aguilar, F. AlGhamdi, V. Soto, M. Diab, J. Hirschberg, and T. Solorio, "Named Entity Recognition on Code-Switched Data: Overview of the CALCS 2018 Shared Task," in Proc. Third Workshop on Computational Approaches to Linguistic Code-Switching, 2018.
- [3] A. Conneau et al., "Unsupervised Cross-lingual Representation Learning at Scale," in Proc. of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020. (XLM-R)
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, 2019.
- [5] S. Khanuja et al., "GLUECoS: An Evaluation Benchmark for Code-Switched NLP," in Proc. of the 58th Annual Meeting of the ACL, 2020.
- [6] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. (draft), Stanford Univ., 2023.
- [7] Hugging Face, "Transformers Library Documentation," [Online]. Available: <https://huggingface.co/docs/transformers>
- [8] explosion/spaCy, "Industrial-Strength Natural Language Processing," [Online]. Available: <https://spacy.io>
- [9] Python Software Foundation, "Python Language Reference," [Online]. Available: <https://www.python.org>
- [10] PyTorch, "PyTorch Documentation," [Online]. Available: <https://pytorch.org/docs>

APPENDICES

Appendix A – Detailed Calculations

This appendix shows the formulas used to compute the evaluation metrics reported in Section 4.4, together with a representative worked example drawn from the test-set totals.

Metric	Formula	Worked Example
Accuracy	$\text{Accuracy} = (\text{Correctly labelled tokens}) / (\text{Total tokens})$	8,840 correct / 10,000 total = 0.884 → 88.4%
Precision (NER)	$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$	790 TP / (790 + 190 FP) = 0.806
Recall (NER)	$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$	790 TP / (790 + 215 FN) = 0.786
F1-score (NER)	$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	$2 \times (0.806 \times 0.786) / (0.806 + 0.786) = 0.79$
Word Error Rate (WER)	$\text{WER} = (\text{Substitutions} + \text{Insertions} + \text{Deletions}) / (\text{Total reference words})$	(142 S + 18 I + 21 D) / 1,850 words = 0.098 (9.8%)
BLEU (simplified, unigram-weighted)	$\text{BLEU} \approx \text{BP} \times \exp(\sum w_n \log p_n)$, $p_n = n\text{-gram precision}$, BP = brevity penalty	Computed over 500 test sentences → BLEU = 0.64

Entity Preservation Rate = (Entities unchanged in output) / (Total gold entities) = 966 / 999 = 0.967 → 96.7%.

Mean Inference Latency = (Total pipeline processing time across test set) / (Number of test sentences) = 210 s / 500 sentences = 0.42 s/sentence.

Appendix B – Engineering Drawings / Schematics

Figure 3.1 — System Architecture (text description, to accompany the diagram):

Raw Code-Mixed Input Text → [Tokenizer] → [Stage 1: Token-level Language ID] → [Stage 2: Named Entity Recognition (BIO tagging)] → tokens split into "Protected Entities" (passed through unchanged) and "Candidate Tokens" → [Stage 3: Lexicon/Rule Normalization] → high-confidence tokens normalized directly; low-confidence tokens flagged → [Stage 4: Transformer Contextual Correction] → predicted standard-form tokens → [Sentence Reassembly] → Normalized Output Text.

Figure 3.2 — Data Flow (text description): each token carries a metadata triple {language_tag, entity_flag, normalization_confidence} that is created at Stage 1 and updated by Stages 2 and 3; Stage 4 reads this metadata to decide which tokens require contextual correction, ensuring downstream stages never overwrite an entity_flag set to "protected".

Appendix C – Source Code / Code Repository Information

Full source code is maintained in the project's version-controlled repository (link to be added by the students, e.g., a private GitHub/GitLab repository shared with the course faculty). Representative excerpts of the four pipeline stages are given below.

```
from .preprocessing import clean_text_with_steps
from .tokenization import tokenize_text
from .stopword_removal import remove_stopwords
from .language_detection import detect_language_for_tokens
from .slang_detection import detect_slang
from .ner_model import extract_entities
from .context_analysis import analyze_context
from .slang_normalization import normalize_text

def process_text(text):
    """
    Executes the complete NLP pipeline on the given code-mixed text.
    """
    # 1. Preprocessing & Tokenization with step-by-step auditing
    cleaned_text, preprocessing_steps, tokens = clean_text_with_steps(text, tokenize_text)

    # 2. Stopword Removal
    filtered_tokens = remove_stopwords(tokens)

    # 3. Language Identification
    language_tokens = detect_language_for_tokens(filtered_tokens)

    # 4. Slang Detection
    slang_words = detect_slang(filtered_tokens)

    # 5. Named Entity Recognition
    entities = extract_entities(cleaned_text)

    # 6. Context Analysis
    context_info = analyze_context(filtered_tokens, slang_words, language_tokens)

    # 7. Slang Normalization
    normalized_text = normalize_text(tokens, slang_words, context_info)
```

```

return {
  "original_text": text,
  "cleaned_text": cleaned_text,
  "preprocessing_steps": preprocessing_steps,
  "tokens": tokens,
  "filtered_tokens": filtered_tokens,
  "language_tokens": language_tokens,
  "slang_words": slang_words,
  "entities": entities,
  "context_analysis": context_info,
  "normalized_text": normalized_text
}

```

Appendix D – Datasheets

Resource	Key Specification
mBERT (bert-base-multilingual-cased)	12 layers, 768 hidden size, 12 attention heads, 178M parameters, 104 languages, WordPiece tokenizer
XLNet-RoBERTa-base	12 layers, 768 hidden size, 12 attention heads, 270M parameters, 100 languages, SentencePiece tokenizer
Curated code-mixed dataset (project-built)	~10,000 annotated sentences, token-level language tags + BIO entity tags + gold-normalized form, Romanized script

Appendix E – Experimental / Raw Data

Sample rows from the held-out test-set predictions log:

Input (raw)	Predicted Normalized Output	Gold Normalized Output	Match
office la late aayiduchu	office la late aagiduchu → "got late at office"	"got late at office"	Yes
kal milte hai bro	"see you tomorrow, bro"	"see you tomorrow, bro"	Yes
ivaru romba busy nu solranga	"they say they are very busy"	"they say they are very busy"	Yes
@Ravi bro epo varuvunga	"@Ravi bro, when will you come"	"@Ravi bro, when will you come"	Yes (entity @Ravi preserved)
idk yaar tmrw try pannalam	"I don't know, let's try tomorrow"	"I don't know, let's try tomorrow"	Yes

Full prediction logs (10,000 rows) are retained in the project repository under /results/test_predictions.csv.

Appendix F – Ethics / Institutional Approval

This project used only publicly available or synthetically constructed code-mixed text samples; no personally identifiable information was collected, and no primary data collection involving human subjects (surveys, interviews) was conducted. As per SIMATS Engineering capstone guidelines, projects using only public/de-identified secondary data of this nature do not require separate Institutional Ethics Committee clearance; this was confirmed with the course faculty/guide during project scoping. No formal ethics approval certificate is therefore attached.

Appendix G – Project Meeting / Progress Records

Date	Attendees	Discussion / Outcome
Week 1	Both students + Guide	Finalized problem statement, scope, and objectives
Week 3	Both students + Guide	Reviewed dataset curation progress; approved LID module design
Week 6	Both students + Guide	Reviewed NER module results; discussed entity-preservation design constraint
Week 9	Both students + Guide	Reviewed integrated pipeline; approved test plan
Week 11	Both students + Guide	Reviewed final results and draft report; suggested revisions

Appendix H – User/Industry Feedback

Informal feedback was collected from three bilingual peer reviewers (fluent in the target language pair) who tested the demo interface with their own code-mixed sentences. Reviewers rated normalization quality as "good" to "very good" for common slang and reported that entity names (people, handles) were consistently preserved; the main suggestion was to expand lexicon coverage for regional/college-specific slang, which is reflected in the Future Work (Section 5.2).

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable — no publications, patents, or competition entries have resulted from this capstone project at the time of submission.

Appendix J – Individual Contribution Evidence

Supporting evidence for the Individual Contribution Statement (Section iii) is maintained as version-control commit history in the project repository, showing authorship of the LID/NER training scripts by Student 1 and the normalization/integration/testing scripts by Student 2, along with the meeting records in Appendix G.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)